

The Linux Command Line

Data Science for Economics

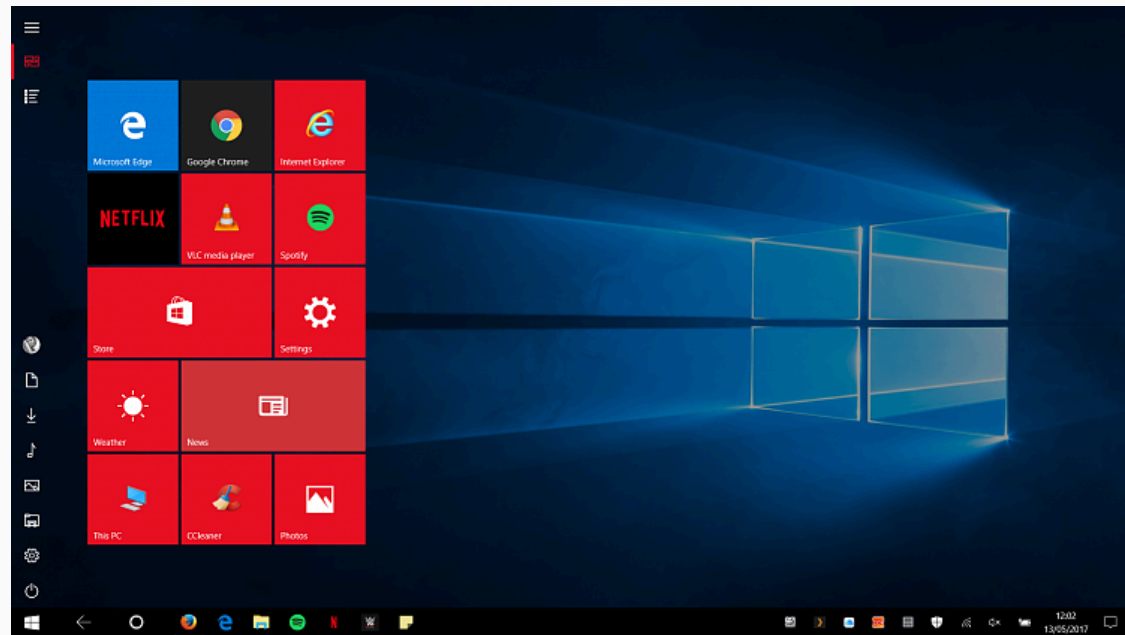
Damian Clarke

dcc213@exeter.ac.uk

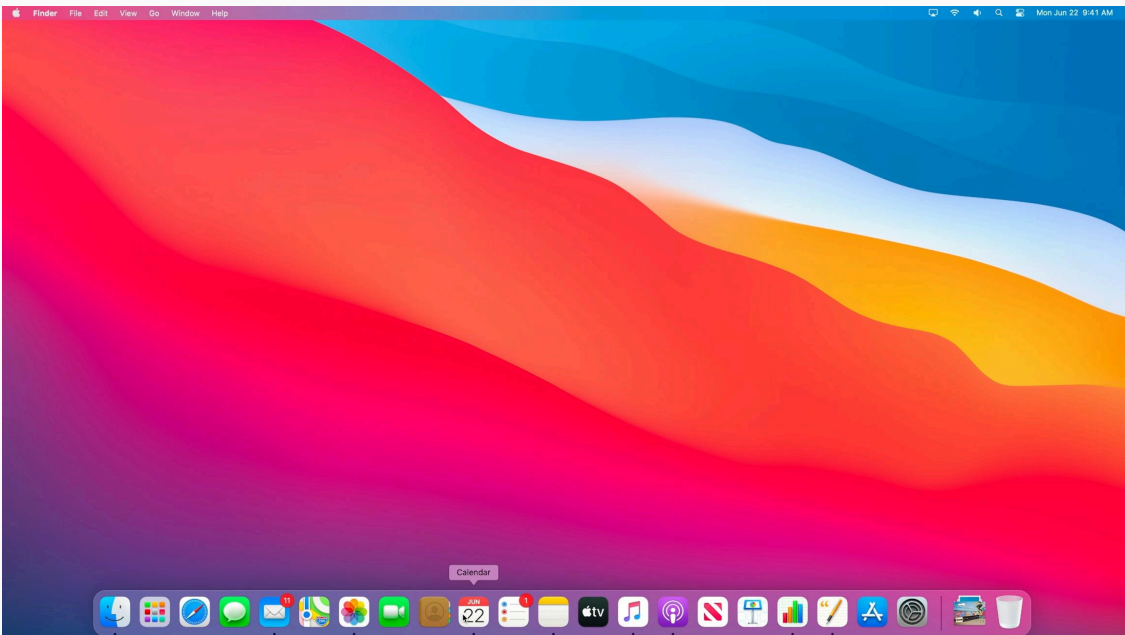
Department of Economics, University of Exeter

16 January, 2029

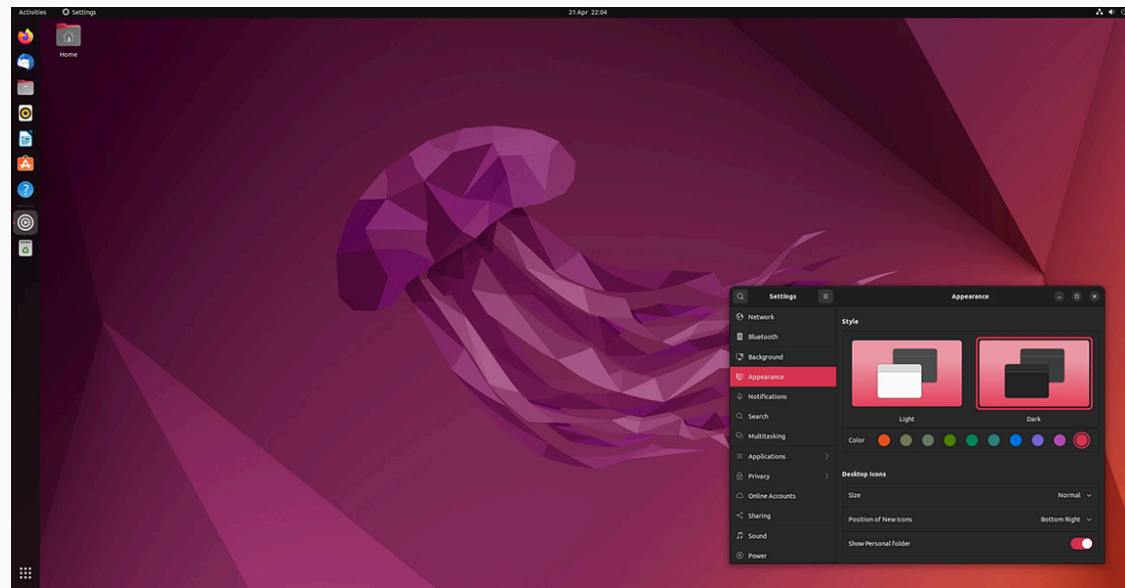
Interfaces and Computational Work



Windows



macOS



Linux (Ubuntu)



“Hacker movies”



The Matrix: Command Line Aesthetic

The Command Line

The Command Line is *also* an interface (“CLI”), but provides a lower level way to interact with your OS

- You are not constrained by the design of graphical user interfaces (GUIs)
- Trade-off between simplicity (GUIs) and flexibility (CLI)
- Also you may hear these related concepts:
 - **The Terminal:** The text input and output environment
 - **The Shell:** The command line interpreter
 - **BASH:** The Linux and Mac default shell (Bourne Again SHell)
 - **The Command Line:** The physical line where commands are typed

Key point: The shell as we will think of it will refer to the BASH version

Windows note: Windows *also* has PowerShell, but **BASH is the industry standard** ✨

Why Learn About the Command Line? 🎯

BASH in particular offers unique advantages:

- **Remarkable flexibility** for OS-level operations
 - Often genuinely the *fastest* way to get tasks done
 - At times I access the command line from directly within other languages because it offers a faster way to do something
- **Helps you understand many other languages**
 - Many command line style functions have been included in other higher-level languages
 - For example: `ls`, `head`, `tail`, `pwd`, `cd`, pipes (`|`), etc.
- **Some OS you will work with *only* have a command line interface**
- **Most servers** you work with will be running a version of BASH
- **Industry expectation:** Data scientists are expected to be comfortable with command line 🧑💼

The Unix Philosophy

The philosophy behind Linux/Unix, espoused by Thompson & Ritchie (early developers):

1. **Write programs that do one thing and do it well**
2. **Write programs to work together**
3. **Write programs to handle text streams**, because that is a universal interface



Thompson & Ritchie

This philosophy drives everything we do in the command line!



A terminal window with a dark background. The title bar at the top reads "damian@damian-paraguay: ~". The terminal content shows the prompt "damian@damian-paraguay:~\$" in green text, followed by a white cursor. The window has standard Linux-style window controls (minimize, maximize, close) in the top right corner.

```
damian@damian-paraguay:~$
```

The Command Line Interface

Understanding the Command Prompt

Generic format:

username@**machine-name**:**directory-location**\$

If “super user” (root):

username@**machine-name**:**directory-location**#

What does this tell you?

- **Blue**: Who you are (your username)
- **Red**: What computer you're on (useful when working remotely!)
- **Orange**: Where you are in the file system
- **\$ vs #**: Regular user vs super user (admin privileges)

Some Simple Commands

Let's start with the basics:

Command	Function	Example
<code>date</code>	Prints out current date and time	<code>date</code>
<code>echo</code> <i>argument</i>	Prints <i>argument</i>	<code>echo "Hello World"</code>
<code>cal</code>	Provides a monthly calendar	<code>cal</code>
<code>man</code> <i>argument</i>	Provides the manual page for <i>argument</i>	<code>man ls</code>

Pro Tip

The `man` command is your friend! Whenever you forget how to use a command, just type `man [command]`. Press `q` to quit the manual page.

In-Class Activity

Try these commands together! Explore `man ls` to see all the different options available.

Navigating and Looking Around

Essential navigation commands:

Command	Function	Example
<code>pwd</code>	Print working directory	<code>pwd</code>
<code>ls</code>	List directory contents	<code>ls -la</code>
<code>cd</code>	Change directory	<code>cd Documents</code>

Key concepts to understand:

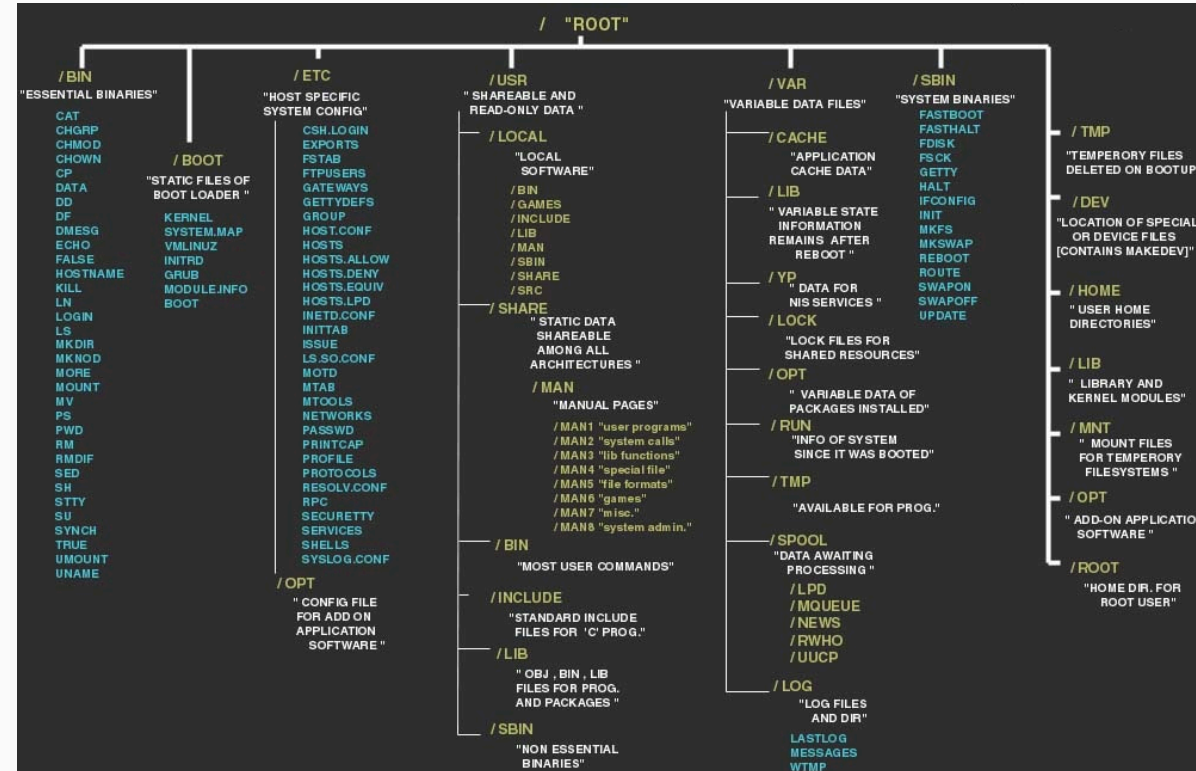
- **Special directories:**

- `.` = current directory
- `..` = parent directory (one level up)
- `~` = your home directory
- `/` = root directory

- **Paths can be relative or absolute:**

- Relative: `cd ../Documents` (relative to where you are)
- Absolute: `cd /home/username/Documents` (full path from root)

Understanding Directory Structures



Linux File System Hierarchy

Important directories to know:

- `/home`: User home directories
- `/etc`: System configuration files
- `/var`: Variable data (logs, etc.)
- `/tmp`: Temporary files
- `/usr`: User programs and data

Exploring Files

Commands for viewing file contents:

Command	Function	Example
<code>cat file</code>	Concatenates files and prints output	<code>cat data.txt</code>
<code>wc file</code>	Provides a word count of file contents	<code>wc -l data.txt</code>
<code>head file</code>	Dumps the first few lines of files	<code>head -n 20 data.txt</code>
<code>tail file</code>	Dumps the last few lines of files	<code>tail -n 10 data.txt</code>

Useful Options

- `head -n 20 file.txt` shows first 20 lines
- `tail -f logfile.txt` follows a file in real-time (great for logs!)
- `wc -l` counts lines, `wc -w` counts words, `wc -c` counts characters

Manipulating Files ⚠

Commands for creating, moving, and deleting:

Command	Function	Example
<code>cp file1 file2</code>	Copies file1 to file2	<code>cp data.txt backup.txt</code>
<code>mv file1 file2</code>	Moves file1 to file2	<code>mv old.txt new.txt</code>
<code>rm file</code>	Eliminates file	<code>rm unwanted.txt</code>
<code>mkdir dir</code>	Creates directory dir	<code>mkdir new_folder</code>

⚠ DANGER ZONE! 💀

You should be **extremely careful** with these commands. A slip of the finger eliminates files (or more!) **permanently**. There is no “Recycle Bin” in the command line!

Especially dangerous: - `rm -rf /` (DO NOT RUN THIS - deletes everything!) - Always double-check before pressing Enter on `rm` commands - Consider using `rm -i` for interactive mode (asks before deleting)

Data Wrangling Commands

Powerful tools for processing data:

Command	Function	Use Case
<code>sort</code> <i>argument</i>	Sorts <i>argument</i> and prints sorted output	Alphabetize data
<code>join</code> <i>file1 file2</i>	Joins lines of files on a common field	Merge datasets
<code>uniq</code> <i>argument</i>	Removes duplicate lines of sorted <i>argument</i>	Find unique values
<code>grep</code>	“Global Regular Expression Print”	Search for patterns
<code>sed</code>	Stream editor	Find and replace
<code>awk</code>	Pattern scanning and text processing language	Complex data manipulation

We will explore these in detail shortly...

Regex in Real Life 🤔



Input/Output Redirection

Key concept: BASH commands can take input and produce output in flexible ways

Inputs can come from:

- Directly as an argument to commands
- From files
- From other commands! 🎉

Outputs can go to:

- Directly printed on screen
- Saved in a file
- Appended to a file
- Passed to other commands! 🎉

This suggests that a series of commands can be sequenced together, omitting any interim input/output

Redirection Operators ⚡

Key elements:

Operator	Function	Example
<	Take input from file	<code>sort < unsorted.txt</code>
>	Send output to file (overwrite)	<code>ls -l > filelist.txt</code>
>>	Append output to file	<code>echo "new line" >> log.txt</code>
	"Pipeline" for concatenating commands	<code>ls -l grep "txt"</code>

Remember

- `>` overwrites the file completely
- `>>` adds to the end of the file
- `|` is one of the most powerful features of the command line!

Practical Example: Counting Files

How many files do we have?

```
ls | wc -l
```

This pipes the list of files to the word count command, counting lines!

In-Class Activity

Navigate to the DATE_FILES directory and try: 1. Count total files with `ls | wc -l` 2. Count files from specific years using wildcards 3. Use `head` and `tail` to explore file naming patterns

How many files from 2017?

```
ls 2017* | wc -l
```

The `*` wildcard matches any characters after “2017”

Combining Files with `cat`

In-Class Activity

1. Create a file combining all data from a specific month
2. Use `wc` to verify the number of lines
3. Compare two files using `diff` to check if they're identical

Example: Combine all January 2017 data

```
cat 2017_01* > Jan_2017.txt
```

This concatenates all files matching the pattern into one file!

Verify the result:

```
wc -l Jan_2017.txt  
head Jan_2017.txt
```


Comparing Files with `diff`

Check if two files are identical:

```
diff file1.txt file2.txt
```

- No output = files are identical
- Output shows differences between files
- `<` indicates lines only in first file
- `>` indicates lines only in second file

Count different lines:

```
diff file1.txt file2.txt | grep "^<" | wc -l
```

Pipeline Examples

Simple search:

```
ls -l | grep "my text"
```

This is equivalent to (but much better than):

```
ls -l > file  
grep < file "my text"  
rm file
```

More complex pipeline:

```
cat data.csv | grep "2024" | sort | uniq | wc -l
```

This: reads a file → filters for 2024 → sorts → removes duplicates → counts lines

Pipelines extend arbitrarily! You can chain as many commands as needed. 🚀

Practical Example: Sorting and Joining

In-Class Activity

Navigate to EXAM_SCORES directory and: 1. Explore the structure of CLASS_1_SCORES.txt and CLASS_2_SCORES.txt 2. Sort both files by name 3. Use `join` to combine scores 4. Try sorting by score instead (use `man sort` to find the right option!)

Scenario: Combine exam scores from two classes

```
# Sort each class file
sort CLASS_1_SCORES.txt > class1_sorted.txt
sort CLASS_2_SCORES.txt > class2_sorted.txt

# Join on student names (first column)
join class1_sorted.txt class2_sorted.txt > joined_scores.txt
```

Result: One file with each student's scores from both classes!

Introduction to Regular Expressions (regex)

What are Regular Expressions?

Regexs are patterns used to match character combinations in strings. “Find” with superpowers!

Why use regex?

- Search for complex patterns, not just exact matches
- Validate input (emails, phone numbers, dates)
- Extract specific information from text, or find and replace with patterns

Basic regex patterns:

Pattern	Matches	Example
.	Any single character	<code>c.t</code> matches cat, cot, cut
*	Zero or more of previous	<code>co*l</code> matches cl, col, cool
+	One or more of previous	<code>co+l</code> matches col, cool (not cl)
?	Zero or one of previous	<code>colou?r</code> matches color, colour
^	Start of line	<code>^The</code> matches “The” at line start
\$	End of line	<code>end\$</code> matches “end” at line end

Regular Expressions: Character Classes

Character classes let you specify sets of characters:

Pattern	Matches	Example
<code>[abc]</code>	Any character in brackets	<code>[aeiou]</code> matches vowels
<code>[^abc]</code>	Any character NOT in brackets	<code>[^0-9]</code> matches non-digits
<code>[a-z]</code>	Any character in range	<code>[A-Z]</code> matches uppercase
<code>\d</code>	Any digit	<code>\d\d\d</code> matches 3 digits
<code>\w</code>	Any word character	<code>\w+</code> matches whole words
<code>\s</code>	Any whitespace	<code>\s+</code> matches spaces/tabs

Quantifiers for repetition:

- `{n}` - exactly n times
- `{n,}` - n or more times
- `{n,m}` - between n and m times

Example: `[0-9]{3}-[0-9]{4}` matches phone numbers like 555-1234

Regular Expressions: Practical Examples

Email validation:

```
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}
```

Date matching (YYYY-MM-DD):

```
[0-9]{4}-[0-9]{2}-[0-9]{2}
```

Finding IP addresses:

```
[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}
```

UK postcode:

```
[A-Z]{1,2}[0-9]{1,2} [0-9][A-Z]{2}
```

Testing Regex

Use online tools like regex101.com to test your patterns before using them in commands!

grep: Global Regular Expression Print

What is grep?

grep searches files for lines matching a pattern and prints them. It's one of the most powerful and commonly used command-line tools!

Basic syntax:

```
grep [options] pattern [file]
```

Common options:

Option	Function	Example
-i	Ignore case	grep -i "error" log.txt
-v	Invert match (show non-matching)	grep -v "comment" file.txt
-n	Show line numbers	grep -n "TODO" script.sh
-c	Count matching lines	grep -c "^#" file.txt
-r	Recursive search in directories	grep -r "function" .
-l	Show only filenames	grep -l "import" *.py
--color	Highlight matches	grep --color "pattern" file.txt

grep Examples in Action

Search for a specific word:

```
grep 'Population' WB_educ_indicators.txt
```

Case-insensitive search with color:

```
grep -i --color=always 'population' WB_educ_indicators.txt
```

Count occurrences:

```
grep -c 'error' logfile.txt
```

Find files containing a pattern:

```
grep -l 'TODO' *.py
```

Combine with other commands:

```
# Find all different commands in your history  
awk '{print $1}' .bash_history | grep "git" | wc -l
```

Regular Expressions with `grep`

In-Class Activity

Using the WB_educ_indicators.txt file: 1. Search for all lines containing “Population ages 0” 2. Count how many times “Female” appears 3. Find all unique indicator types using `grep + sort + uniq`

Using regex patterns in `grep`:

```
# Find lines starting with a digit
grep '^[0-9]' data.txt

# Find email addresses
grep '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' contacts.txt

# Find dates in format YYYY-MM-DD
grep '[0-9]{4}-[0-9]{2}-[0-9]{2}' events.txt

# Find lines ending with specific words
grep 'error$' logfile.txt

# Find lines with exactly 3 digits
grep '\b[0-9]{3}\b' numbers.txt
```

Introduction to `sed`: Stream Editor

What is sed?

`sed` (Stream EDitor) is a powerful text manipulation tool that processes text line by line. It's perfect for: - Find and replace operations - Deleting lines - Inserting text - Transforming text

Basic syntax:

```
sed 's/pattern/replacement/' file
```

The 's' command (substitute) is the most common sed operation: - `s/old/new/` - replace first occurrence - `s/old/new/g` - replace all occurrences (global) - `s/old/new/2` - replace second occurrence - `s/old/new/gi` - replace all, case-insensitive

sed: Common Operations

Find and replace:

```
# Replace first occurrence per line
sed 's/cat/dog/' file.txt

# Replace all occurrences (global)
sed 's/cat/dog/g' file.txt

# Case-insensitive replacement
sed 's/error/ERROR/gi' log.txt

# Replace and save to new file
sed 's/old/new/g' input.txt > output.txt

# Replace in-place (modify original file)
sed -i 's/old/new/g' file.txt
```

Delete operations:

```
# Delete lines containing pattern
sed '/pattern/d' file.txt

# Delete blank lines
sed '/^$/d' file.txt

# Delete lines 5-10
sed '5,10d' file.txt
```


sed: Advanced Examples

Multiple operations:

```
# Chain multiple substitutions
sed 's/cat/dog/g; s/mouse/rat/g' file.txt

# Or use multiple -e flags
sed -e 's/cat/dog/g' -e 's/mouse/rat/g' file.txt
```

Using delimiters:

```
# When pattern contains '/', use different delimiter
sed 's|/home/user|/home/newuser|g' paths.txt
```

Capturing groups:

```
# Swap first two words on each line
sed 's/\([^ ]*\) \([^ ]*\)/\2 \1/' file.txt

# Extract dates from format "Date: 2024-01-15"
sed 's/Date: \([0-9-]*\)\/\1/' file.txt
```

Line insertion:

```
# Insert text before line 3
sed '3i\This is inserted text' file.txt

# Append text after matching line
sed '/pattern/a\New line after match' file.txt
```

Introduction to `awk`: Pattern Processing Language 🎓

What is `awk`?

`awk` is a complete programming language designed for text processing. It's named after its creators: Aho, Weinberger, and Kernighan.

Why use `awk`? - Process structured data (columns/fields) - Perform calculations on data - Generate reports
- Complex pattern matching and text transformation

Basic structure:

```
awk 'pattern { action }' file
```

Key concepts: - Automatically splits each line into **fields** (columns) - `$1`, `$2`, `$3`, ... represent field 1, field 2, field 3, etc. - `$0` represents the entire line - Default field separator is whitespace (spaces/tabs)

awk: Basic Usage

Print specific columns:

```
# Print first column
awk '{print $1}' file.txt

# Print multiple columns
awk '{print $1, $3}' file.txt

# Print with custom formatting
awk '{print "Name:", $1, "Score:", $2}' scores.txt
```

Changing field separator:

```
# Use semicolon as separator
awk -F ";" '{print $1}' data.csv

# Use comma (for CSV files)
awk -F "," '{print $1, $3}' data.csv

# Multiple character separator
awk -F "::" '{print $2}' file.txt
```

Pattern matching:

```
# Print lines containing pattern
awk '/error/ {print}' logfile.txt

# Print specific column only for matching lines
awk '/2024/ {print $1, $5}' data.txt
```

awk: Advanced Features

Built-in variables:

```
# NR = Number of Records (line number)
awk '{print NR, $0}' file.txt

# NF = Number of Fields (columns in current line)
awk '{print NF, $0}' file.txt

# Print last column of each line
awk '{print $NF}' file.txt
```

Conditions and calculations:

```
# Print lines where column 3 > 100
awk '$3 > 100 {print}' data.txt

# Calculate sum of column 2
awk '{sum += $2} END {print sum}' numbers.txt

# Calculate average
awk '{sum += $2; count++} END {print sum/count}' numbers.txt

# Multiple conditions
awk '$3 > 50 && $4 < 100 {print $1, $3}' data.txt
```

awk: Practical Examples

Working with delimited data:

```
# Extract specific columns from CSV
awk -F ";" '{print $2, $11}' WB_educ_indicators.txt

# Print with custom output separator
awk -F ";" 'BEGIN {OFS=","} {print $1, $2, $3}' input.txt
```

Statistical operations:

```
# Find maximum value in column 2
awk 'BEGIN {max = 0} {if ($2 > max) max = $2} END {print max}' data.txt

# Count lines matching condition
awk '/error/ {count++} END {print count}' logfile.txt

# Group and count
awk '{count[$1]++} END {for (word in count) print word, count[word]}' file.txt
```

Combining patterns:

```
# Multiple patterns with different actions
awk '/^#/ {comments++} /^$/ {blanks++} END {print "Comments:", comments, "Blanks:", blanks}' script.sh
```

Real-World Example: World Bank Data

Exploring the data structure:

```
# Check file size
ls -lh WB_educ_indicators.txt

# Count total lines
wc -l WB_educ_indicators.txt

# View first few lines
head -n 3 WB_educ_indicators.txt

# View column headers
head -n 1 WB_educ_indicators.txt
```

Extract specific information:

```
# Get unique indicator names
awk -F ";" '{print $11}' WB_educ_indicators.txt | sort | uniq

# Count unique countries
awk -F ";" '{print $2}' WB_educ_indicators.txt | sort | uniq | wc -l

# Extract population data to new file
grep 'Population ages 0' WB_educ_indicators.txt > population_data.txt
```

Combining Everything: Power Pipeline

Find your most-used commands:

```
# Step by step build-up
awk '{print $1}' .bash_history          # Get first word
awk '{print $1}' .bash_history | sort  # Sort them
awk '{print $1}' .bash_history | sort | uniq -c # Count unique
awk '{print $1}' .bash_history | sort | uniq -c | sort -n -r # Sort by frequency
awk '{print $1}' .bash_history | sort | uniq -c | sort -n -r | head # Top 10
```

This demonstrates: - Start simple, build complexity - Each step does one thing well - Pipe output from one command to input of next - The Unix philosophy in action! ✨

In-Class Activity

Explore your own bash history: 1. Find your 10 most-used commands 2. Count how many times you've used `ls` versus `cd` 3. Find all commands containing a specific word

Advanced Example: Analyzing Exam Scores

Task: Find top-performing students across both classes

```
# Sort by score (second column, numerically)
sort -t' ' -k2 -n -r CLASS_1_SCORES.txt | head -n 5

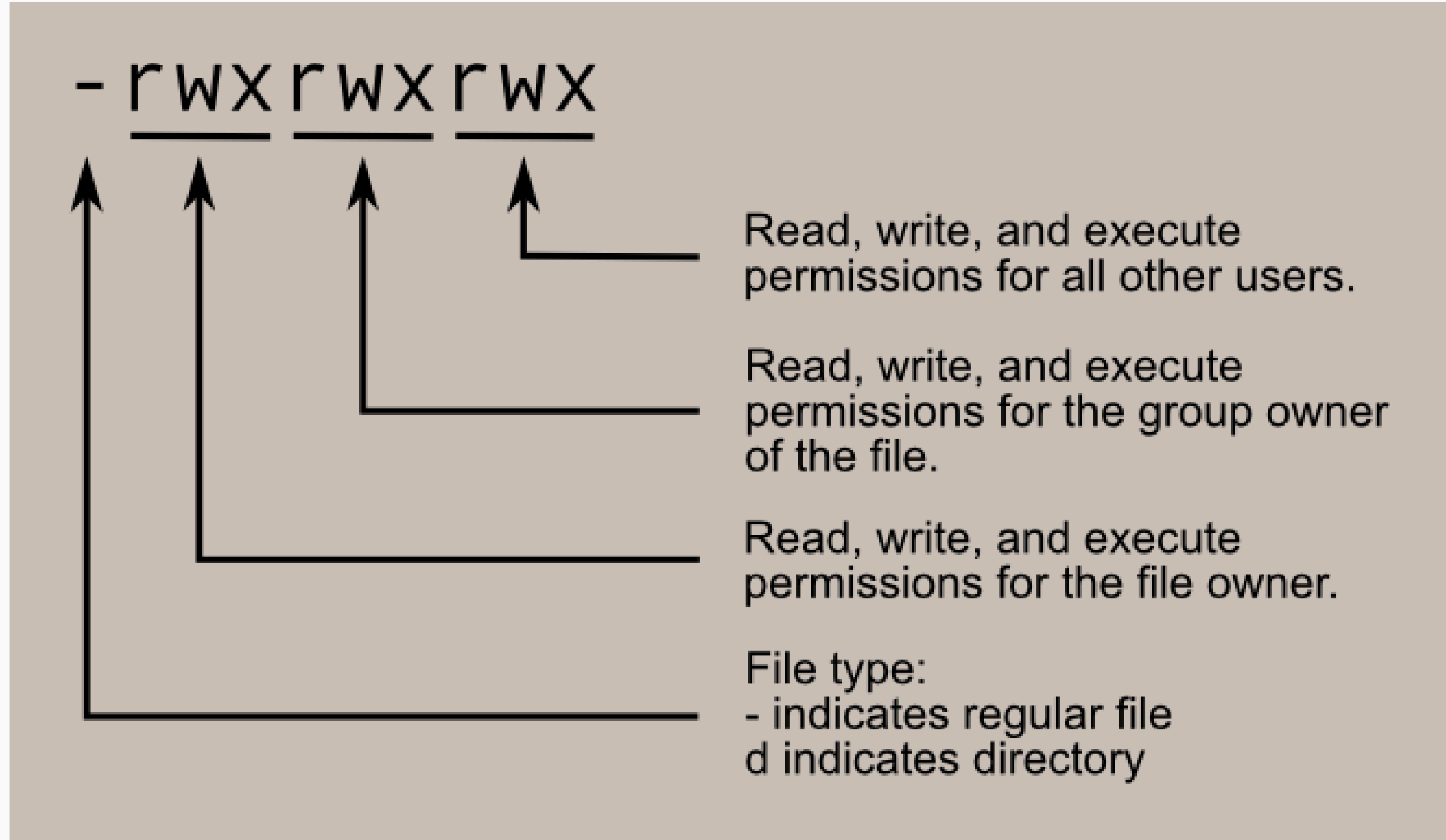
# Combine classes and find overall top performers
cat CLASS_1_SCORES.txt CLASS_2_SCORES.txt | \
  sort -t' ' -k2 -n -r | head -n 10

# Calculate average score per class
awk '{sum += $2; count++} END {print "Average:", sum/count}' CLASS_1_SCORES.txt
```

Using awk for more complex analysis:

```
# Find students who scored above 80 in both classes
join class1_sorted.txt class2_sorted.txt | \
  awk '$2 > 80 && $3 > 80 {print $1, "Class1:", $2, "Class2:", $3}'
```

Users and Permissions 🔑



File Permissions Diagram

Source: linuxcommand.org

Understanding Permissions

View permissions with `ls -l`

Permission commands:

Command	Function	Example
<code>chmod</code>	Modify rights for a file	<code>chmod +x script.sh</code>
<code>su</code>	Become the super user temporarily	<code>su -</code>
<code>sudo</code>	Execute command as super user	<code>sudo apt update</code>
<code>chown</code>	Change a file's owner	<code>chown user:group file.txt</code>
<code>chgrp</code>	Change a file's group	<code>chgrp staff file.txt</code>

Common chmod patterns

- `chmod +x file` - make executable
- `chmod 755 file` - owner can do everything, others can read/execute
- `chmod 644 file` - owner can read/write, others can only read

Writing Your First Bash Script!

If we do certain processes regularly, we should write scripts!

Let's create a simple script together:

```
#!/bin/bash

# A simple script to print hello world!
echo "Hello world!"
```

To make it executable:

```
chmod +x myscript.sh
./myscript.sh
```

Script Best Practices

1. Always start with `#!/bin/bash` (the “shebang”)
2. Comment your code with `#`
3. Use meaningful names for your scripts
4. Make scripts executable with `chmod +x`
5. Test thoroughly before running on important data!

Useful Bash Script Example

A script to analyze data files:

```
#!/bin/bash

# Script to process World Bank data

echo "Processing World Bank education indicators..."

# Extract population data
grep 'Population ages 0' WB_educ_indicators.txt > population_data.txt

# Count unique countries
countries=$(awk -F ";" '{print $2}' WB_educ_indicators.txt | sort | uniq | wc -l)

echo "Found data for $countries countries"
echo "Population data saved to population_data.txt"
```

In-Class Activity

Create your first bash script that: 1. Lists files in current directory 2. Counts them 3. Prints a summary message

Where to From Here

Next steps:

1. **Make sure your command line environment is properly set up** (please refer to last unit's notes)

- **Linux or Mac users:** quite simple (perhaps Ctrl+Alt+T or Cmd+Space → Terminal)
- **Windows users:** WSL installation (we covered this last week!)

2. **Practice, practice, practice!**

- The command line feels awkward at first
- With practice, it becomes second nature
- Start using it for everyday tasks

3. **Additional resources:**

- Multiple references on ELE
- A modern book to work along with: “[The Linux Command Line](#)”
- Classic reference: “The Unix Programming Environment”
- Online regex tester: regex101.com

Practice Exercises for This Week

Basic exercises:

1. Navigate to your home directory and create a folder called `bee2041_practice`
2. Create a file inside called `test.txt` with some text
3. Copy this file to `test_backup.txt`
4. List all files and check their permissions
5. Use `grep` to search for a word in your file
6. Try chaining commands with pipes: `ls -l | grep ".txt" | wc -l`

Advanced challenges:

7. Use `awk` to extract and sum numbers from a data file
8. Create a bash script that automates file organization
9. Use `sed` to find and replace patterns in multiple files
10. Analyze the `WB_educ_indicators.txt` file to find specific insights

Key Takeaways 🎓

1. The command line is a powerful interface that gives you direct control over your OS
2. BASH is the industry standard and essential for data science work
3. Core concepts: navigation (`cd`, `ls`, `pwd`), file manipulation (`cp`, `mv`, `rm`), and viewing (`cat`, `head`, `tail`)
4. Pipelines (`|`) let you chain commands together for powerful data processing
5. Always be careful with destructive commands like `rm`!
6. Practice makes perfect - the more you use it, the more natural it becomes

Remember: The command line is your friend, not your enemy! 🤝